

AULA 10 – PORTAL DE COMUNICAÇÃO SI
PAINEL ADMINISTRATIVO (08/10/26)

SUMÁRIO

1. Objetivos

2. Fundamentação Teórica

- 2.1 O Painel Administrativo e o Padrão CRUD
- 2.2 PDO (PHP Data Objects) com PostgreSQL
- 2.3 Prevenção Crítica contra SQL Injection
- 2.4 Mecanismos de Upload Seguro de Arquivos
- 2.5 Gerenciamento Dinâmico de Conteúdo Multimídia
- 2.6 Camadas de Segurança e Mitigação de XSS

3. Arquitetura e Estrutura do Painel Administrativo

- 3.1 Estrutura de Diretórios e Arquivos
- 3.2 Folha de Estilo e Layout Responsivo em CSS Puro

4. Conexão com PostgreSQL via Camada PDO

5. Implementação Completa do CRUD de Conteúdos

- 5.1 Modelagem Relacional (DDL)
- 5.2 Listagem de Registros (conteudos_listar.php)
- 5.3 Cadastro e Processamento (conteudos_cadastrar.php)
- 5.4 Atualização de Dados (conteudos_editar.php)
- 5.5 Exclusão Segura (conteudos_excluir.php)

6. Pipeline de Upload Seguro de Imagens

7. Diretrizes de Segurança no Ambiente Administrativo

8. Guia de Estudos e Verificação Conceitual

9. Roteiro Prático de Desenvolvimento em Equipe

10. Exercícios de Fixação Teórico-Práticos

11. Checklist de Entrega da Etapa 1 e Cronograma da Etapa 2

REFERÊNCIAS

1. OBJETIVOS DE APRENDIZAGEM

Esta aula capacita as equipes de desenvolvimento na consolidação da área restrita de gestão do portal institucional. A manipulação de dados de forma segura e performática representa a espinha dorsal de qualquer sistema corporativo moderno, demandando rigor técnico no manuseio de registros persistidos e arquivos em disco.

Ao término deste módulo, o acadêmico estará apto a:

- Estruturar a arquitetura física e lógica de um painel administrativo (*dashboard*) utilizando padrões limpos e sem frameworks;
- Implementar operações completas de criação, leitura, atualização e exclusão (*CRUD*) integradas ao PostgreSQL por meio da extensão PDO;
- Desenvolver pipelines robustos de upload de imagens no servidor, aplicando tripla validação (código de erro, tamanho em bytes e validação real de tipo MIME via extensões binárias);
- Blindar a aplicação contra vulnerabilidades críticas categorizadas pelo OWASP Top 10, especificamente *SQL Injection* e *Cross-Site Scripting* (XSS), utilizando consultas preparadas (*prepared statements*) e escape contextual com `htmlspecialchars`.

2. FUNDAMENTAÇÃO TEÓRICA

Bons estudos!

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

2.1 O PAINEL ADMINISTRATIVO E O PADRÃO CRUD

O painel administrativo funciona como o núcleo operacional de um Sistema de Gerenciamento de Conteúdo (CMS). Trata-se de uma interface protegida por credenciais de autenticação em que usuários autorizados operam sobre o ciclo de vida da informação (Welling & Thomson, 2017). Essas operações consolidam-se no acrônimo **CRUD**: *Create* (inserção), *Read* (leitura/recuperação), *Update* (atualização) e *Delete* (exclusão lógica ou física). No ecossistema web, cada operação reflete verbos semânticos do protocolo HTTP (POST, GET, POST simulando PUT/DELETE) mapeados diretamente para instruções relacionais do padrão SQL (INSERT, SELECT, UPDATE, DELETE).

2.2 PDO (PHP DATA OBJECTS) COM POSTGRESQL

O **PDO** representa uma camada de abstração para acesso a bancos de dados relacionais no ecossistema PHP (PHP Documentation Group, 2026a). Diferente dos drivers legados monomotores (como o módulo obsoleto `pg_*` em implementações procedurais diretas), o PDO disponibiliza uma interface uniforme orientada a objetos compatível com dezenas de Sistemas Gerenciadores de Banco de Dados Relacionais (SGBDs).

A inicialização de uma instância PDO exige a definição de um Data Source Name (DSN), string que encapsula o driver de conexão (neste projeto, exclusivamente `pgsql`), o endereço de rede do servidor (host), a porta de escuta do serviço (port=5432) e a base de dados de destino (dbname). A manipulação de exceções é centralizada sob a classe especializada `PDOException`, ativada explicitamente pela constante de configuração `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`, eliminando falhas silenciosas em tempo de execução (GoalKicker, 2018).

2.3 PREVENÇÃO CRÍTICA CONTRA SQL INJECTION

A vulnerabilidade de **Injeção de SQL** (SQL Injection) ocorre quando dados fornecidos por clientes não confiáveis (vindos de `$_GET`, `$_POST` ou cookies) são interpolados diretamente na estrutura de uma string de consulta enviada ao interpretador do SGBD. Nesses cenários, comandos maliciosos injetam operadores relacionais ou delimitadores (como aspas simples e comandos de terminação `;`), redefinindo a árvore sintática da instrução e expondo dados sensíveis ou permitindo destruição catastrófica de tabelas.

A mitigação definitiva e obrigatória consiste no uso de **Consultas Preparadas** (*Prepared Statements*). Esse mecanismo desacopla o ciclo de vida da instrução SQL em duas fases:

- **Fase de Compilação/Otimização:** A string SQL contendo delimitadores de parâmetros nomeados (como `:id` ou `:titulo`) é submetida ao PostgreSQL via método `prepare()`. O motor compila, gera o plano de execução e bloqueia a sintaxe da consulta;
- **Fase de Vinculação e Execução:** Os dados são atrelados aos parâmetros via `bindParam()`, `bindValue()` ou array associativo no método `execute()`. O banco trata todo e qualquer dado exclusivamente como literal/conteúdo, neutralizando qualquer tentativa de interpretação semântica de código embutido.

2.4 MECANISMOS DE UPLOAD SEGURO DE ARQUIVOS

A submissão de artefatos binários por formulários web requer o atributo `enctype="multipart/form-data"` na tag `<form>`. Em tempo de requisição, o interpretador PHP intercepta o fluxo de bytes e popula o array global multidimensional `$_FILES`, contendo informações cruciais: `name` (nome original na estação cliente), `type` (cabeçalho MIME fornecido pelo navegador), `size` (tamanho em bytes), `tmp_name` (caminho temporário alocado no sistema operacional do servidor) e `error` (código de status do processamento).

Atenção Crítica de Segurança: O índice `$_FILES['imagem']['type']` é uma informação montada pelo navegador cliente. Ele **NUNCA** deve ser utilizado para validar o tipo real de um arquivo, pois pode ser trivialmente adulterado via interceptação HTTP (burp suite, curl). A verificação deve ser obrigatoriamente realizada no servidor através da extensão `FileInfo` (função `finfo_file`) inspecionando os números mágicos (magic bytes) do cabeçalho binário (PHP Documentation Group, 2026b).

O arquivo aprovado no pipeline nunca deve preservar seu nome original — o que abriria margem para ataques de sobrescrita de arquivos e Directory Traversal. O arquivo deve ser renomeado via algoritmo gerador de identificadores únicos determinísticos (como `uniqid()` combinado com extensões sanitizadas) e transferido de sua área transitória para a pasta pública por meio da função nativa estrita `move_uploaded_file()`, garantindo a integridade operacional da camada de I/O.

2.5 GERENCIAMENTO DINÂMICO DE CONTEÚDO MULTIMÍDIA

A comunicação institucional moderna baseia-se em narrativas dinâmicas que alternam blocos textuais extensos, identificação categórica, imagens destacadas e mídias de vídeo externas (YouTube). No âmbito da arquitetura de dados, a persistência de arquivos binários dentro do próprio banco (via tipos BLOB ou BYTEA) é estritamente desencorajada para imagens de portais comuns devido à degradação de cache, gargalo de memória RAM no SGBD e aumento desnecessário no tamanho dos backups relacionais. O padrão arquitetural consolidado consiste no armazenamento físico do arquivo no subsistema de arquivos (*filesystem* local) e na gravação estrita de sua rota relativa no campo relacional (VARCHAR).

2.6 CAMADAS DE SEGURANÇA E MITIGAÇÃO DE XSS

O ataque de **Cross-Site Scripting (XSS)** manifesta-se quando scripts maliciosos (geralmente JavaScript) são inseridos em campos de entrada e, posteriormente, renderizados diretamente pelo navegador de terceiros sem a devida sanitização. Em ambientes administrativos, um ataque XSS armazenado (*Stored XSS*) pode permitir o sequestro instantâneo da sessão do administrador via leitura forçada de cookies desprotegidos.

A regra fundamental da defesa em profundidade determina: *filtre as entradas, escape as saídas*. Toda exibição de dados dinâmicos armazenados no banco de dados nas páginas de visualização HTML deve passar obrigatoriamente pelo método de escape `htmlspecialchars($dado, ENT_QUOTES, 'UTF-8')`, neutralizando a interpretação de caracteres reservados da sintaxe HTML (<, >, &, " e ').

3. ESTRUTURA DO PAINEL ADMINISTRATIVO

3.1 ESTRUTURA DE DIRETÓRIOS E ARQUIVOS

A modularização do projeto obedece ao princípio da responsabilidade única, isolando a área restrita na subpasta `/admin`. Esta abordagem impede que códigos administrativos compartilhem escopo público inadvertidamente.

```
portal_si/  
├── admin/  
│   ├── css/  
│   │   └── admin.css  
│   ├── uploads/  
│   │   └── .gitkeep  
│   ├── includes/  
│   │   ├── conexao.php  
│   │   ├── verifica_sessao.php  
│   │   ├── header.php  
│   │   └── footer.php  
│   ├── conteudos_cadastrar.php  
│   ├── conteudos_editar.php  
│   ├── conteudos_excluir.php  
│   ├── conteudos_listar.php  
│   ├── index.php  
│   └── logout.php  
└── index.php  
... (arquivos públicos do portal)
```

3.2 FOLHA DE ESTILO E LAYOUT RESPONSIVO EM CSS PURO

Bons estudos!

Em cumprimento rigoroso aos requisitos do projeto (que veda expressamente bibliotecas CSS externas como Bootstrap, Tailwind ou utilitários JS), a interface administrativa é estilizada em CSS3 puro estruturado com CSS Grid e Flexbox.

Salvar como admin/css/admin.css:

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif; } body { background-color: #f4f6f9; color: #333333; display: flex; min-height: 100vh; }
/*Sidebar / Menu Lateral*/
.sidebar { width: 250px; background-color: #1a252f; color: #ffffff; padding: 24px 16px; display: flex; flex-direction: column; }
.sidebar h2 { font-size: 18px; margin-bottom: 24px; color: #3498db; text-transform: uppercase; letter-spacing: 1px; }
.sidebar nav ul { list-style: none; }
.sidebar nav ul li { margin-bottom: 8px; }
.sidebar nav ul li a { color: #bdc3c7; text-decoration: none; display: block; padding: 10px 14px; border-radius: 4px; transition: background-color 0.2s, color 0.2s; }
.sidebar nav ul li a:hover, .sidebar nav ul li a.active { background-color: #34495e; color: #ffffff; }
.sidebar .user-info { margin-top: auto; padding-top: 16px; border-top: 1px solid #34495e; font-size: 13px; color: #95a5a6; }
/*Área Principal de Conteúdo*/
.main-content { flex: 1; padding: 32px; overflow-y: auto; }
.header-panel { display: flex; justify-content: space-between; align-items: center; margin-bottom: 28px; }
.header-panel h1 { font-size: 24px; color: #2c3e50; }
/*Cards do Dashboard*/
.cards-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 20px; margin-bottom: 32px; }
.card { background: #ffffff; padding: 24px; border-radius: 6px; box-shadow: 0 1px 3px rgba(0,0,0,0.08); border-left: 4px solid #3498db; }
.card h3 { font-size: 13px; color: #7f8c8d; text-transform: uppercase; margin-bottom: 8px; }
.card .card-value { font-size: 28px; font-weight: 700; color: #2c3e50; }
/*Tabelas Administrativas*/
.table-container { background: #ffffff; border-radius: 6px; box-shadow: 0 1px 3px rgba(0,0,0,0.08); overflow-x: auto; }
table { width: 100%; border-collapse: collapse; text-align: left; font-size: 14px; }
thead th { background-color: #f8f9fa; color: #6c757d; padding: 14px 16px; border-bottom: 2px solid #e9ecef; font-weight: 600; }
tbody td { padding: 14px 16px; border-bottom: 1px solid #e9ecef; vertical-align: middle; }
tbody tr:hover { background-color: #f8f9fa; }
/*Formulários*/
.form-container { background: #ffffff; padding: 28px; border-radius: 6px; box-shadow: 0 1px 3px rgba(0,0,0,0.08); max-width: 800px; }
.form-group { margin-bottom: 20px; }
.form-group label { display: block; margin-bottom: 6px; font-weight: 600; font-size: 14px; color: #2c3e50; }
.form-control { width: 100%; padding: 10px 12px; border: 1px solid #ced4da; border-radius: 4px; font-size: 14px; color: #495057; }
.form-control:focus { border-color: #80bdff; outline: none; box-shadow: 0 0 0 0.2rem rgba(0,123,255,0.25); }
textarea.form-control { min-height: 140px; resize: vertical; }
/*Botões*/
.btn { display: inline-block; font-weight: 500; text-align: center; padding: 8px 16px; font-size: 13px; border-radius: 4px; text-decoration: none; cursor: pointer; border: 1px solid transparent; transition: background-color 0.15s; }
.btn-primary { background-color: #3498db; color: #fff; }
.btn-primary:hover { background-color: #2980b9; }
.btn-success { background-color: #2ecc71; color: #fff; }
.btn-success:hover { background-color: #27ae60; }
.btn-danger { background-color: #e74c3c; color: #fff; }
.btn-danger:hover { background-color: #c0392b; }
.btn-sm { padding: 4px 8px; font-size: 12px; }
/*Mensagens de Alerta*/
.alert { padding: 12px 16px; margin-bottom: 20px; border-radius: 4px; font-size: 14px; }
.alert-success { background-color: #d4edda; color: #155724; border: 1px solid #c3e6cb; }
.alert-danger { background-color: #f8d7da; color: #721c24; border: 1px solid #f5c6cb; }
```

4. CONEXÃO COM POSTGRESQL VIA CAMADA PDO

O arquivo de infraestrutura conexao.php centraliza a conexão relacional. Ele utiliza o driver pgsql, configura o disparo estrito de exceções e define a recuperação padrão de registros como matriz associativa, prevenindo duplicidade de índices numéricos na memória.

Salvar como admin/includes/conexao.php:

```
<?php
/** Conexão com o Banco de Dados PostgreSQL via PDO */
$host = 'localhost';
$port = '5432';
$dbname = 'portal_si';
$usuario = 'postgres';
$senha = 'postgres'; // Ajuste conforme credenciais do ambiente de laboratório

$dsn = "pgsql:host={$host};port={$port};dbname={$dbname}";

try {
    $pdo = new PDO($dsn, $usuario, $senha, [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                                            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                                            PDO::ATTR_EMULATE_PREPARES => false
                                          ]);
} catch (PDOException $e) {
    // Em produção, nunca exponha $e->getMessage() diretamente ao usuário
    error_log("Erro de Conexão PDO: " . $e->getMessage());
    die("Falha crítica na conexão com o banco de dados. Contate o administrador.");
}
```

5. IMPLEMENTAÇÃO COMPLETA DO CRUD DE CONTEÚDOS

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

5.1 MODELAGEM RELACIONAL (DDL)

Para garantir integridade referencial estrita e manter conformidade com as restrições arquiteturais, execute o script DDL no terminal psql ou em sua ferramenta visual conectada ao PostgreSQL.

```
-- Criação da tabela de categorias de conteúdos
CREATE TABLE IF NOT EXISTS categorias ( id SERIAL PRIMARY KEY, nome VARCHAR(100) NOT NULL UNIQUE );

-- Carga inicial de categorias institucionais
INSERT INTO categorias (nome) VALUES ('Notícias da ESUCRI'), ('Eventos Acadêmicos'), ('Estágios e Oportunidades'), ('Artigos e Tutoriais') ON CONFLICT (nome) DO NOTHING;

-- Criação da tabela principal de conteúdos
CREATE TABLE IF NOT EXISTS conteudos ( id SERIAL PRIMARY KEY, titulo VARCHAR(200) NOT NULL, id_categoria INTEGER NOT NULL REFERENCES categorias(id) ON DELETE RESTRICT, texto TEXT NOT NULL, imagem VARCHAR(255), video_url VARCHAR(255), data_publicacao TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP, id_usuario INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT );

-- Índices operacionais para aceleração de filtros e ordenação
CREATE INDEX IF NOT EXISTS idx_conteudos_data ON conteudos(data_publicacao DESC); CREATE INDEX IF NOT EXISTS idx_conteudos_categoria ON conteudos(id_categoria);
```

5.2 LISTAGEM DE REGISTROS (CONTEUDOS_LISTAR.PHP)

A listagem realiza a junção de projeções (JOIN) entre conteúdos, categorias e usuários autores, viabilizando auditoria e visualização limpa com botões de manipulação direta.

```
<?php
require_once __DIR__ . '/includes/verifica_sessao.php';
require_once __DIR__ . '/includes/conexao.php';

// Busca registros unindo tabelas de referência
$sql = "SELECT c.id, c.titulo, c.data_publicacao, c.imagem,
            cat.nome AS categoria_nome, u.nome AS autor_nome
        FROM conteudos c
        INNER JOIN categorias cat ON c.id_categoria = cat.id
        INNER JOIN usuarios u ON c.id_usuario = u.id
        ORDER BY c.data_publicacao DESC";

$stmt = $pdo->query($sql);
$conteudos = $stmt->fetchAll();

require_once __DIR__ . '/includes/header.php';
?>

<div class="header-panel">
    <h1>Gestão de Conteúdos</h1>
    <a href="conteudos_cadastrar.php" class="btn btn-primary">+ Novo Conteúdo</a>
</div>

<?php if (isset($_GET['sucesso'])): ?>
    <div class="alert alert-success">Operação realizada com sucesso!</div>
<?php elseif (isset($_GET['erro'])): ?>
    <div class="alert alert-danger"><?= htmlspecialchars($_GET['erro'], ENT_QUOTES, 'UTF-8') ?></div>
<?php endif; ?>

<div class="table-container">
    <table>
    <thead>
    <tr>
    <th style="width: 50px;">ID</th>
    <th style="width: 80px;">Capa</th>
    <th>Titulo</th>
    <th>Categoria</th>
    <th>Autor</th>
    <th>Data</th>
    <th style="width: 140px; text-align: center;">Ações</th>
    </tr>
    </thead>
    <tbody>
    <?php if (count($conteudos) === 0): ?>
    <tr>
    <td colspan="7" style="text-align: center; color: #7f8c8d;">Nenhum registro encontrado.</td>
    </tr>
    <?php else: ?>
    <?php foreach ($conteudos as $item): ?>
    <tr>
    <td><?= (int)$item['id'] ?></td>
    <td>
    <?php if (!empty($item['imagem']) && file_exists(__DIR__ . '/' . $item['imagem'])): ?>
    
    <?php else: ?>
    <span style="font-size: 11px; color: #bdc3c7;">Sem foto</span>
    <?php endif; ?>
    </td>
    </tr>
    </tbody>
    </table>
</div>
```

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

```

<td><strong><?= htmlspecialchars($item['titulo'], ENT_QUOTES, 'UTF-8') ?></strong></td>
<td><?= htmlspecialchars($item['categoria_nome'], ENT_QUOTES, 'UTF-8') ?></td>
<td><?= htmlspecialchars($item['autor_nome'], ENT_QUOTES, 'UTF-8') ?></td>
<td><?= date('d/m/Y H:i', strtotime($item['data_publicacao'])) ?></td>
<td style="text-align: center;">
<a href="conteudos_editar.php?id=<?= (int)$item['id'] ?>" class="btn btn-primary btn-sm">Editar</a>
<a href="conteudos_excluir.php?id=<?= (int)$item['id'] ?>" class="btn btn-danger btn-sm" onclick="return confirm('Deseja
realmente excluir este conteúdo permanentemente?');">Excluir</a>
</td>
</tr>
<?php endforeach; ?>
<?php endif; ?>
</tbody>
</table>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

5.3 CADASTRO E PROCESSAMENTO (CONTEUDOS_CADASTRAR.PHP)

O script trata a renderização do formulário e seu processamento seguro, realizando a vinculação dos parâmetros de inserção e retornando o ID do registro gerado no PostgreSQL pela cláusula RETURNING id.

```

$erros = []; // Carrega categorias para o combo
$catQuery = $pdo->query("SELECT id, nome FROM categorias ORDER BY nome ASC");
$categorias = $catQuery->fetchAll(); if ($SERVER['REQUEST_METHOD'] === 'POST') {
    $titulo = trim($POST['titulo'] ?? '');
    $id_categoria = (int)($POST['id_categoria'] ?? 0);
    $texto = trim($POST['texto'] ?? '');
    $video_url = trim($POST['video_url'] ?? '');
    $id_usuario = (int)$SESSION['usuario_id'];
    $caminho_imagem = null;
    // Validações no servidor
    if (empty($titulo)) {
        $erros[] = "O campo Título é obrigatório.";
    } elseif (mb_strlen($titulo) > 200) {
        $erros[] = "O título não pode exceder 200 caracteres.";
    }

    if ($id_categoria <= 0) { $erros[] = "Selecione uma categoria válida."; }

    if (empty($texto)) { $erros[] = "O campo Texto/Conteúdo é obrigatório."; }

    // Tratamento de Upload de Imagem
    if (isset($FILES['imagem']) && $FILES['imagem']['error'] !== UPLOAD_ERR_NO_FILE) {
        if ($FILES['imagem']['error'] === UPLOAD_ERR_OK) {
            $tamanho_maximo = 2 * 1024 * 1024; // 2 Megabytes
            if ($FILES['imagem']['size'] > $tamanho_maximo) {
                $erros[] = "A imagem selecionada excede o limite estrito de 2 MB.";
            }

            // Validação real de MIME Type
            $finfo = new finfo(FILEINFO_MIME_TYPE);
            $mime_detectado = $finfo->file($FILES['imagem']['tmp_name']);
            $mimes_permitidos = [
                'image/jpeg' => 'jpg',
                'image/pjpeg' => 'jpg',
                'image/png' => 'png',
                'image/gif' => 'gif'
            ];

            if (!array_key_exists($mime_detectado, $mimes_permitidos)) {
                $erros[] = "Formato de imagem inválido. São aceitos apenas arquivos JPG, PNG ou GIF.";
            }

            if (empty($erros)) {
                $extensao = $mimes_permitidos[$mime_detectado];
                $nome_arquivo = 'conteudo_' . uniqid('', true) . '.' . $extensao;
                $destino_diretorio = __DIR__ . '/uploads/';

                if (!is_dir($destino_diretorio)) {
                    mkdir($destino_diretorio, 0755, true);
                }

                $destino_completo = $destino_diretorio . $nome_arquivo;

                if (move_uploaded_file($FILES['imagem']['tmp_name'], $destino_completo)) {
                    $caminho_imagem = 'uploads/' . $nome_arquivo;
                } else {
                    $erros[] = "Falha ao gravar arquivo de imagem no disco.";
                }
            }
        } else {
            $erros[] = "Falha ao mover o arquivo de imagem para o destino.";
        }
    }
}

```

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

```

        $erros[] = "Erro operacional no envio da imagem. Código: " . $_FILES['imagem']['error'];
    }
}

// Persistência caso não existam inconsistências
if (empty($erros)) {
    $sql = "INSERT INTO conteudos (titulo, id_categoria, texto, imagem, video_url, id_usuario)
        VALUES (:titulo, :id_categoria, :texto, :imagem, :video_url, :id_usuario)
        RETURNING id";

    try {
        $stmt = $pdo->prepare($sql);
        $stmt->execute([
            ':titulo'      => $titulo,
            ':id_categoria' => $id_categoria,
            ':texto'       => $texto,
            ':imagem'      => $caminho_imagem,
            ':video_url'   => !empty($video_url) ? $video_url : null,
            ':id_usuario'  => $id_usuario
        ]);

        header("Location: conteudos_listar.php?sucesso=1");
        exit;
    } catch (PDOException $e) {
        error_log("Erro de inserção: " . $e->getMessage());
        $erros[] = "Falha ao persistir registro no banco de dados.";
    }
}
}
require_once DIR . '/includes/header.php';
?>

```

Publicar Novo Conteúdo

Voltar à Listagem

```

<div class="alert alert-danger">
    <ul>
        <?php foreach ($erros as $erro): ?>
            <li><?= htmlspecialchars($erro, ENT_QUOTES, 'UTF-8') ?></li>
        <?php endforeach; ?>
    </ul>
</div>

```

Título do Conteúdo

```

<div class="form-group">
    <label for="id_categoria">Categoria Acadêmica *</label>
    <select name="id_categoria" id="id_categoria" class="form-control" required>
        <option value="">Selecione uma categoria...</option>
        <?php foreach ($categorias as $cat): ?>
            <option value="<?= (int)$cat['id'] ?>"
                <?= (isset($_POST['id_categoria']) && (int)$_POST['id_categoria'] === (int)$cat['id']) ? 'selected' : '' ?>>
                <?= htmlspecialchars($cat['nome'], ENT_QUOTES, 'UTF-8') ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>

<div class="form-group">
    <label for="texto">Corpo da Publicação (Texto Integral) *</label>
    <textarea name="texto"
        id="texto"
        class="form-control" required><?= htmlspecialchars($_POST['texto'] ?? '', ENT_QUOTES, 'UTF-8') ?>
    </textarea>
</div>

<div class="form-group">
    <label for="imagem">Imagem Destaque (Capa - Máx 2MB, JPG/PNG)</label>
    <input type="file" name="imagem" id="imagem" class="form-control" accept="image/jpeg,image/png,image/gif">
</div>

<div class="form-group">
    <label for="video_url">URL de Vídeo do YouTube (Exemplo: https://www.youtube.com/watch?v=...)</label>
    <input type="url" name="video_url" id="video_url" class="form-control" placeholder="https://www.youtube.com/watch?v=..."
    value="<?= htmlspecialchars($_POST['video_url'] ?? '', ENT_QUOTES, 'UTF-8') ?>>
</div>

<button type="submit" class="btn btn-success">Salvar e Publicar</button>
</form>

```

5.4 ATUALIZAÇÃO DE DADOS (CONTEUDOS_EDITAR.PHP)

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

A rotina de edição exige carregamento seguro do registro por ID validado numericamente. Havendo novo envio de imagem, a anterior — se existente — deve ser expurgada do disco via unlink() para evitar arquivos órfãos.

```
<?php
require_once __DIR__ . '/includes/verifica_sessao.php';
require_once __DIR__ . '/includes/conexao.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if (!$id) {
    header("Location: conteudos_listar.php?erro=Identificador+inválido");
    exit;
}

// Busca o registro atual
$stmt = $pdo->prepare("SELECT * FROM conteudos WHERE id = :id");
$stmt->execute([':id' => $id]);
$conteudo = $stmt->fetch();

if (!$conteudo) {
    header("Location: conteudos_listar.php?erro=Registro+não+encontrado");
    exit;
}

$catQuery = $pdo->query("SELECT id, nome FROM categorias ORDER BY nome ASC");
$categorias = $catQuery->fetchAll();

$erros = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $titulo = trim($_POST['titulo'] ?? '');
    $id_categoria = (int)($_POST['id_categoria'] ?? 0);
    $texto = trim($_POST['texto'] ?? '');
    $video_url = trim($_POST['video_url'] ?? '');
    $caminho_imagem = $conteudo['imagem']; // Mantém o caminho atual por padrão

    if (empty($titulo)) {
        $erros[] = "O campo Título é obrigatório.";
    }
    if ($id_categoria <= 0) { $erros[] = "Selecione uma categoria válida."; }
    if (empty($texto)) { $erros[] = "O campo Texto é obrigatório."; }

    // Processamento de upload de substituição
    if (isset($_FILES['imagem']) && $_FILES['imagem']['error'] !== UPLOAD_ERR_NO_FILE) {
        if ($_FILES['imagem']['error'] === UPLOAD_ERR_OK) {
            $tamanho_maximo = 2 * 1024 * 1024;
            if ($_FILES['imagem']['size'] > $tamanho_maximo) { $erros[] = "A nova imagem excede o teto de 2 MB."; }

            $finfo = new finfo(FILEINFO_MIME_TYPE);
            $mime_detectado = $finfo->file($_FILES['imagem']['tmp_name']);
            $mimes_permitidos = [
                'image/jpeg' => 'jpg',
                'image/pjpeg' => 'jpg',
                'image/png' => 'png',
                'image/gif' => 'gif'
            ];

            if (!array_key_exists($mime_detectado, $mimes_permitidos)) {
                $erros[] = "Formato inválido na substituição de imagem.";
            }

            if (empty($erros)) {
                $extensao = $mimes_permitidos[$mime_detectado];
                $nome_arquivo = 'conteudo_' . uniqid('', true) . '.' . $extensao;
                $destino_completo = __DIR__ . '/uploads/' . $nome_arquivo;

                if (move_uploaded_file($_FILES['imagem']['tmp_name'], $destino_completo)) {
                    // Remove arquivo antigo fisicamente se ele existir
                    if (!empty($conteudo['imagem']) && file_exists(__DIR__ . '/' . $conteudo['imagem'])) {
                        unlink(__DIR__ . '/' . $conteudo['imagem']);
                    }
                    $caminho_imagem = 'uploads/' . $nome_arquivo;
                } else { $erros[] = "Falha ao gravar substituição de imagem."; }
            }
        } else { $erros[] = "Falha no envio da nova imagem. Código: " . $_FILES['imagem']['error']; }
    }

    if (empty($erros)) {
        $sql = "UPDATE conteudos
            SET titulo = :titulo, id_categoria = :id_categoria, texto = :texto,
                imagem = :imagem, video_url = :video_url
            WHERE id = :id";

        try {
```

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido

```

$updateStmt = $pdo->prepare($sql);
$updateStmt->execute([
    ':titulo' => $titulo,
    ':id_categoria' => $id_categoria,
    ':texto' => $texto,
    ':imagem' => $caminho_imagem,
    ':video_url' => !empty($video_url) ? $video_url : null,
    ':id' => $id
]);

header("Location: conteudos_listar.php?sucesso=1");
exit;
} catch (PDOException $e) {
    error_log("Erro de atualização: " . $e->getMessage());
    $erros[] = "Erro operacional ao atualizar o banco de dados.";
}
}
}

require_once __DIR__ . '/includes/header.php';
?>

<div class="header-panel">
    <h1>Editar Conteúdo #<?=(int)$id ?></h1>
    <a href="conteudos_listar.php" class="btn btn-primary">Voltar</a>
</div>

<?php if (!empty($erros)): ?>
    <div class="alert alert-danger">
        <ul>
            <?php foreach ($erros as $erro): ?>
                <li><?=(int)$erro ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<div class="form-container">
    <form action="conteudos_editar.php?id=<?=(int)$id ?>" method="POST" enctype="multipart/form-data">
        <div class="form-group">
            <label for="titulo">Título *</label>
            <input type="text" name="titulo" id="titulo" class="form-control" maxlength="200" required value="<?=(int)$titulo ?>" />
        </div>

        <div class="form-group">
            <label for="id_categoria">Categoria Acadêmica *</label>
            <select name="id_categoria" id="id_categoria" class="form-control" required>
                <?php foreach ($categorias as $cat): ?>
                    <?php
                        $catSelecionada = (isset($_POST['id_categoria'])) ? (int)$_POST['id_categoria'] : (int)$catSelecionada;
                    <?>
                    <option value="<?=(int)$cat['id'] ?>" <?=(int)$catSelecionada === (int)$cat['id'] ? 'selected' : '' ?>>
                        <?=(int)$cat['nome'] ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>

        <div class="form-group">
            <label for="texto">Texto / Artigo Completo *</label>
            <textarea name="texto" id="texto" class="form-control" required><?=(int)$texto ?></textarea>
        </div>

        <div class="form-group">
            <label for="imagem">Substituir Imagem (Deixe vazio para manter a atual)</label>
            <?php if (!empty($_POST['imagem'])): ?>
                <div style="margin-bottom: 8px;">
                    
                </div>
            <?php endif; ?>
            <input type="file" name="imagem" id="imagem" class="form-control" accept="image/jpeg,image/png,image/gif" />
        </div>

        <div class="form-group">
            <label for="video_url">URL do YouTube</label>
            <input type="url" name="video_url" id="video_url" class="form-control" value="<?=(int)$video_url ?>" />
        </div>

        <button type="submit" class="btn btn-success">Salvar Alterações</button>
    </form>
</div>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>5.5 Exclusão Segura (conteudos_excluir.php)
```

5.5 EXCLUSÃO SEGURA (CONTEUDOS_EXCLUIR.PHP)

A rotina de exclusão precisa inspecionar e remover o arquivo de imagem alocado em disco antes de descartar a tupla na base de dados, garantindo higiene no sistema de arquivos.

```
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if (!$id) {
    header("Location: conteudos_listar.php?erro=Identificador+inválido");
    exit;
}try {
    // 1. Localiza a imagem vinculada
    $stmt = $pdo->prepare("SELECT imagem FROM conteudos WHERE id = :id");
    $stmt->execute([':id' => $id]);
    $registro = $stmt->fetch();
    if ($registro) {
        // 2. Remove o arquivo fisico se existir
        if (!empty($registro['imagem'])) {
            $arquivoFisico = __DIR__ . '/' . $registro['imagem'];
            if (file_exists($arquivoFisico)) {
                unlink($arquivoFisico);
            }
        }

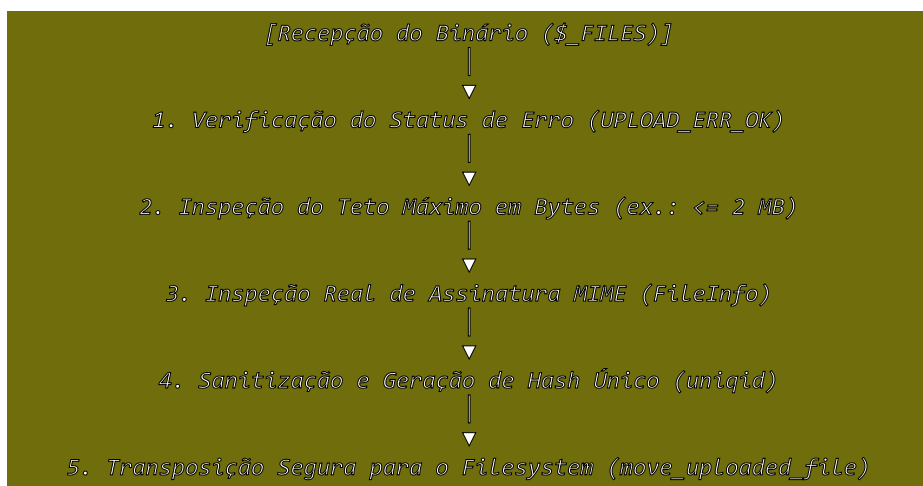
        // 3. Deleta o registro relacional via comando parametrizado
        $deleteStmt = $pdo->prepare("DELETE FROM conteudos WHERE id = :id");
        $deleteStmt->execute([':id' => $id]);

        header("Location: conteudos_listar.php?sucesso=1");
        exit;
    } else {
        header("Location: conteudos_listar.php?erro=Registro+inexistente");
        exit;
    }
} catch (PDOException $e) {
    error_log("Erro ao excluir: " . $e->getMessage());
    header("Location: conteudos_listar.php?erro=Impossível+excluir+o+registro");
    exit;
}
```

6. PIPELINE DE UPLOAD SEGURO DE IMAGENS

O envio de artefatos binários por usuários representa um dos vetores de ataque mais críticos em aplicações web. Vetores comuns incluem ataques de inclusão remota/local de arquivos (LFI/RFI) e injeção de executáveis no servidor web (como um arquivo malicioso shell.php mascarado de foto).

Um pipeline de upload seguro deve seguir estas cinco etapas sequenciais:



- **Inspeção de Erro de Transporte:** O campo \$_FILES['campo']['error'] deve ser avaliado estritamente contra a constante nativa UPLOAD_ERR_OK (código inteiro 0). Casos como UPLOAD_ERR_INI_SIZE ou UPLOAD_ERR_PARTIAL devem abortar imediatamente o processamento;

- **Inspecção de Limites Físicos:** Validação estrita do índice \$_FILES['campo']['size']. No caso deste projeto, estipulamos 2.097.152 bytes (\$2 \times 1024 \times 1024\$), prevenindo esgotamento de disco ou ataques DoS de armazenamento;
- **Análise de Números Mágicos (MIME Type):** Nunca confiar na extensão informada no nome do arquivo original ou no parâmetro \$_FILES['campo']['type']. O uso da biblioteca nativa finfo lê os bytes de cabeçalho do arquivo temporário alocado;
- **Renomeação Determinística:** O nome original do cliente nunca deve ser mantido. Utilizamos a expressão uniqid('conteudo_', true) que gera identificadores baseados no relógio de microssegundos somado a entropia aleatória;
- **Gravação e Isolamento:** Aplica-se move_uploaded_file(), garantindo que o arquivo tenha sido legitimamente transferido via POST. Salva-se apenas a URI relativa no PostgreSQL (ex.: uploads/conteudo_67041a.jpg), nunca referências absolutas do sistema operacional hospedeiro.

7. DIRETRIZES DE SEGURANÇA NO AMBIENTE ADMINISTRATIVO

7.1 GUARDIÃO DE SESSÃO CENTRALIZADO

O arquivo admin/includes/verifica_sessao.php deve ser importado na primeira linha de execução de todas as páginas administrativas. Ele atua como um portão de segurança que interrompe o ciclo de vida do script caso o usuário não esteja devidamente autenticado.

Salvar como admin/includes/verifica_sessao.php:

```
<?php
/**
 * Guardiã de Acesso Administrativo
 */
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Verifica a existência do identificador validado no login
if (!isset($_SESSION['usuario_id']) || empty($_SESSION['usuario_id'])) {
    // Destrói quaisquer vestígios residuais
    session_unset();
    session_destroy();

    // Redireciona com cabeçalho de status para o formulário de login
    header("Location: ../login.php?erro=Acesso+restrito.+Efetue+login.");
    exit;
}

// Opcional: Renovação periódica do ID de sessão para mitigar Session Fixation
if (!isset($_SESSION['ultimo_acesso']) || (time() - $_SESSION['ultimo_acesso'] > 1800)) {
    session_regenerate_id(true);
    $_SESSION['ultimo_acesso'] = time();
}
```

7.2 PROTEÇÃO RIGOROSA CONTRA CROSS-SITE SCRIPTING (XSS)

Conforme consolidado nas seções anteriores, qualquer saída textual renderizada dentro de nós do DOM ou atributos HTML oriunda da base de dados ou parâmetros de URL **deve ser tratada** via função nativa:

```
echo htmlspecialchars($dado, ENT_QUOTES, 'UTF-8');
```

O parâmetro ENT_QUOTES é essencial: ele escapa tanto aspas duplas (") quanto aspas simples ('), impedindo que usuários mal-intencionados quebrem delimitadores de atributos HTML durante a navegação.

7.3 VALIDAÇÃO DE ENTRADA NO LADO DO SERVIDOR

Validações do lado do cliente (como atributos required, maxlength e type="email" do HTML5) existem exclusivamente para oferecer usabilidade ao operador. Elas são facilmente burladas com desativação do interpretador JavaScript no navegador ou montagem manual de pacotes HTTP. Portanto, **toda e qualquer validação deve ser reprocessada integralmente no PHP** antes de qualquer tentativa de persistência no PostgreSQL.

Bons estudos!

8. GUIA DE ESTUDOS E VERIFICAÇÃO CONCEITUAL

Resumo dos Pontos-Chave da Aula

- O CRUD é o alicerce operacional dos painéis administrativos, mapeando rotinas lógicas a comandos do padrão relacional SQL;
- A extensão PDO padroniza o acesso a dados no PHP, gerencia conexões via DSN com tratamento robusto de exceções e atua como barreira contra injeções SQL;
- *Prepared Statements* (consultas preparadas) isolam as instruções estruturais dos dados fornecidos pelo usuário, eliminando a manipulação indevida da árvore sintática da instrução;
- Uploads seguros de arquivos exigem validação de erro em \$_FILES, controle de tamanho em bytes, identificação de tipo MIME real via FileInfo e renomeação aleatória no disco;
- As saídas em telas públicas ou restritas demandam tratamento com htmlspecialchars(), anulando ameaças de XSS refletido ou persistido.

Perguntas de Verificação

1. Por que a concatenação direta de variáveis em strings SQL (ex.: "WHERE id = " . \$_GET['id']) é uma falha grave de segurança?

Resposta Esperada: Porque permite ao usuário injetar operadores ou comandos SQL maliciosos que reescrevem a lógica original da consulta compilada pelo PostgreSQL, expondo ou destruindo dados sensíveis.

2. O que representa e armazena a superglobal \$_FILES durante uma requisição HTTP do tipo POST?

Resposta Esperada: Armazena uma matriz multidimensional com os metadados dos arquivos enviados através do formulário: o nome original na máquina cliente (name), o MIME alegado pelo cliente (type), o tamanho em bytes (size), o caminho temporário no disco do servidor (tmp_name) e o código de erro retornado pela submissão (error).

3. Por quais motivos técnicos e operacionais nunca devemos salvar um arquivo de upload com o nome original fornecido pelo usuário?

Resposta Esperada: Para prevenir ataques de *Path Traversal* (tentativa de navegar na árvore de diretórios com sequências ../), evitar a sobrescrita acidental de arquivos com o mesmo nome e neutralizar a execução de arquivos com nomes maliciosos ou caracteres conflitantes com o sistema operacional do servidor.

9. ROTEIRO PRÁTICO DE DESENVOLVIMENTO EM EQUIPE

As atividades desta aula prática devem ser realizadas pelas equipes de 4 acadêmicos constituídas para o Projeto de Extensão IV. Sigam o passo a passo de desenvolvimento:

- **Configuração de Diretórios e Permissões:** Criar a subpasta /admin na raiz do portal;
- Criar o subdiretório /admin/uploads e garantir que o processo do servidor web tenha permissões de escrita em disco (chmod 755 ou equivalente no ambiente operacional);
- Criar o arquivo /admin/css/admin.css com o layout base fornecido.
- **Infraestrutura de Banco e Conexão:** Executar o script DDL na base PostgreSQL (portal_si) para criação das tabelas categorias e conteudos;
- Implementar o arquivo admin/includes/conexao.php e validar a conexão via terminal ou página de teste;
- Implementar admin/includes/verifica_sessao.php e incluí-lo em todas as páginas da área /admin.
- **Construção do CRUD de Conteúdos:** Montar o arquivo conteudos_listar.php, estruturando a tabela visual com os botões operacionais;
- Desenvolver o arquivo conteudos_cadastrar.php, implementando as validações obrigatórias no formulário e no processamento do upload;

Bons estudos!

- Implementar a rotina `conteudos_editar.php`, assegurando a pré-carga correta das informações e a substituição limpa da imagem em disco;
- Criar `conteudos_excluir.php`, validando a exclusão do registro no banco combinada com a remoção física da imagem via `unlink()`.
- **Homologação e Testes de Fluxo:** Autenticar-se via módulo de login com as credenciais cadastradas;
- Inserir um conteúdo completo preenchendo todos os campos e anexando uma imagem;
- Tentar enviar um arquivo inválido (como um script `.txt` ou `.php`) para validar se o bloqueio do `FileInfo` funciona conforme o esperado;
- Testar a atualização e exclusão do conteúdo cadastrado, checando a persistência na base e no diretório físico de imagens.

10. EXERCÍCIOS DE FIXAÇÃO TEÓRICO-PRÁTICOS

Exercício 1 — Identificação de Falha Crítica

Analise criticamente o bloco de código PHP abaixo, retirado de uma versão legada:

```
<?php // Código com vulnerabilidade grave $id = $_GET['id']; $resultado = $pdo->query("SELECT * FROM conteudos WHERE id = " . $id); $dados = $resultado->fetch(); echo "<h1>" . $dados['titulo'] . "</h1>"; ?>
```

Identifique as duas falhas de segurança presentes e apresente a versão corrigida do código.

Gabarito:

As duas falhas são:

- **Injeção de SQL (SQL Injection):** A variável `$id` proveniente da URL (`$_GET`) foi concatenada diretamente na query SQL sem sanitização ou uso de prepared statement;
- **Cross-Site Scripting (XSS):** O dado `$dados['titulo']` foi impresso na tela sem o tratamento com `htmlspecialchars()`.

Correção:

```
<?php $id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT); if (!$id) { die("ID inválido."); } $stmt = $pdo->prepare("SELECT * FROM conteudos WHERE id = :id"); $stmt->execute([':id' => $id]); $dados = $stmt->fetch(); if ($dados) { echo "<h1>" . htmlspecialchars($dados['titulo'], ENT_QUOTES, 'UTF-8') . "</h1>"; } ?>
```

Exercício 2 — Validação de Upload

Escreva uma função modularizada em PHP denominada `validarImagemUpload(array $arquivo)`: array que receba o array `$_FILES['campo']` e retorne uma lista de erros encontrados. Devem ser validadas as seguintes regras:

- Verificação de erro no upload (`UPLOAD_ERR_OK`);
- Limite máximo de 2 MB (2.097.152 bytes);
- Validação real de MIME type com `finfo` (aceitar apenas `image/jpeg` e `image/png`).

Gabarito:

```
function validarImagemUpload(array $arquivo): array { $erros = []; if ($arquivo['error'] !== UPLOAD_ERR_OK) { $erros[] = "Falha no envio da imagem. Código de erro: " . $arquivo['error']; return $erros; } $maxBytes = 2 * 1024 * 1024; if ($arquivo['size'] > $maxBytes) { $erros[] = "O arquivo excede o limite estrito de 2 MB."; } $finfo = new finfo(FILEINFO_MIME_TYPE); $mime = $finfo->file($arquivo['tmp_name']); $mimesValidos = ['image/jpeg', 'image/png']; if (!in_array($mime, $mimesValidos, true)) { $erros[] = "Tipo de arquivo não permitido. Apenas JPG e PNG são aceitos."; } return $erros; }
```

Exercício 3 — Escrita de Consulta Parametrizada

Escreva o fragmento de código PHP utilizando PDO para realizar a busca de todos os conteúdos pertencentes a uma categoria informada por parâmetro na URL (`$_GET['categoria_id']`), exibindo o título de cada publicação. A consulta deve ser devidamente protegida contra SQL Injection e garantir tipagem estrita de dados.

Gabarito:

Bons estudos!

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

```
<?php $categoriaId = filter_input(INPUT_GET, 'categoria_id', FILTER_VALIDATE_INT);if ($categoriaId) { $stmt = $pdo->prepare("SELECT titulo FROM conteudos WHERE id_categoria = :cat_id ORDER BY id DESC"); $stmt->execute(['cat_id' => $categoriaId]); $registros = $stmt->fetchAll();foreach ($registros as $linha) { echo "<p>". htmlspecialchars($linha['titulo'], ENT_QUOTES, 'UTF-8') . "</p>"; } } else { echo "<p>Identificador de categoria inválido.</p>"; } ?>
```

11. CHECKLIST DE ENTREGA DA ETAPA 1 E CRONOGRAMA DA ETAPA 2

Fica estabelecido o alinhamento das etapas do Projeto de Extensão IV — Portal de Comunicação SI. A **Etapa 1** compõe a primeira nota formal da disciplina; o acompanhamento das entregas garante o cumprimento dos prazos previstos.

Checklist de Avaliação da Etapa 1 (Entrega em Andamento)

- * Modelagem e criação das tabelas no banco de dados PostgreSQL estruturadas com integridade referencial;
- * Sistema completo de autenticação (cadastro seguro de usuários com criptografia via password_hash e verificação com password_verify);
- * Fluxo de login e logout com inicialização e destruição correta de sessões HTTP;
- * Layout institucional público padronizado (header, menu, área central e footer) em HTML5 semântico e CSS3 puro.

Cronograma para Apresentação Final (Etapa 2)

Data	Aula	Marco Operacional
15/10/2026	Aula extra	
22/10/2026	Aula 11	Integração e Exibição Dinâmica (consumo dos registros do PostgreSQL na Home Page e seções do portal)
29/10/2026	Aula 12	Finalização e Testes de Carga, Validação Funcional, Usabilidade e Blindagem de Segurança
05/11/2026	Aula 13	Preparação para Apresentação (alinhamento dos grupos, validação final da hospedagem e ensaio técnico)
12/11/2026	Aula 14	Apresentação Final e Entrega da Etapa 2 (Avaliação do projeto produzido)

Importante: Em virtude do recesso acadêmico do Dia do Professor em 15/10 ter sido transferido para o dia 13/10, teremos as atividades letivas no dia 15/10. O cronograma não comporta extensões de prazo: a entrega e apresentação final da Etapa 2 dar-se-á impreterivelmente na data estipulada de 12/11/2026.

REFERÊNCIAS

GOALKICKER.COM. **PHP Notes for Professionals**. Londres: GoalKicker, 2018. Disponível em: <https://goalkicker.com/PHPBook/>. Acesso em: 07 out. 2026.

PHP DOCUMENTATION GROUP. **PHP Manual: PHP Data Objects (PDO)**. The PHP Group, 2026a. Disponível em: https://www.php.net/manual/pt_BR/book.pdo.php. Acesso em: 07 out. 2026.

PHP DOCUMENTATION GROUP. **PHP Manual: Handling file uploads**. The PHP Group, 2026b. Disponível em: https://www.php.net/manual/pt_BR/features.file-upload.php. Acesso em: 07 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. PostgreSQL, 2026. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 07 out. 2026.

WELLING, Luke; THOMSON, Laura. **PHP e MySQL: Desenvolvimento Web**. 5. ed. Rio de Janeiro: Alta Books, 2017.

Chamada1

Chamada2

Chamada3

Define_Verde

Define_Amarelo

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido

	FACULDADES ESUCRI – Criciúma – SC			
	Curso:	Sistemas de Informação	2026/2	
	Disciplina:	Projeto de Extensão IV		
	Professor:	Esp. Jucemar Formigoni Cândido	jucemar.candido@esucri.com.br	

Estrutura

Código

Bons estudos!

Material preparado para a disciplina de **Projeto de Extensão IV – ESUCRI 2026/2** | Professor: Jucemar Formigoni Cândido